Optimal Locations for Transmitters on a Circular Road for Interference Minimization

Anshuman Sharma(bmat2406) Shyamal Borgohain(bmat2431)
bmat2406@isibang.ac.in bmat2431@isibang.ac.in
Indian Statistical Institute, Bangalore Centre

Abstract

A circular road has n check posts and m possible locations for transmitters. Each transmitter can serve all check posts within 250 m along the shortest arc of the circle. The goal is to ensure that every check post is served by at least one transmitter while minimizing the number of check posts served by more than one transmitter.

Firstly, we present an algorithm that uses dynamic programming to find the optimal locations which runs in $\mathcal{O}(m^3 + m \log n + n \log n)$. Later, we present an algorithm that uses a sliding-window technique and monotonic-queue along with dynamic programming to improve efficiency. Its time complexity is $\mathcal{O}(m^2 + m \log n + n \log n)$, and its space complexity is $\mathcal{O}(n + m)$. Also, both the algorithms return the selected transmitter locations.

1 Problem Statement

There is a circular road around a military building complex and there are n check posts built around the perimeter of the circular road. For efficient communication between check posts, it is decided to built transmitters along the road so that every check-post has a transmitter within 250m of it. (The distance is measured as the length of the shortest arc between two points along the perimeter of the circle) m possible locations are identified where transmitters can be built. However, in order to avoid signal interference, it is recommended that there exists exactly one transmitter within 250m of a check post. Give an algorithm to find the optimal locations for transmitters such that the number of check-posts that has more than one transmitters within 250m of it is minimized.

1.1 Formulation

Let us consider a circular road of circumference L. Suppose there are n distinct check-posts at positions $p_0 < p_1 < \cdots < p_{n-1}$ in $[0, L)$ and m candidate transmitter locations $c_1, \ldots, c_m \in [0, L)$. Let $r = 250\,\text{m}$ and let $d(x, y)$ be the length of the shorter arc between x and y. A check post p can receive signals from a transmitter at location c if $d(c, p) \leq r$.

For a set X of locations for transmitters, let $\text{cnt}_X(p)$ be the number of locations of X serving p. X is *feasible* if $\text{cnt}_X(p) \geq 1$ for every post p, and its cost is

$$cost(X) = \big|\{p : \ \text{cnt}_X(p) \geq 2\}\big|$$

Aim. Find a feasible X of minimum cost, or report that none exists. (A feasible set exists iff every post is served by at least one candidate, since $X =$ all candidates is then feasible.)

If $2r \geq L$, every site serves every post, so any single site gives cost 0. From now on assume $2r < L$.

2 Preliminaries and Structural Observations

2.1 Unrolling and arcs

Due to circular geometry transmitters near the perimeter boundary wrap around and cover check-posts at the beginning of the and vice-versa. To resolve this problem, we unwrap the circle by conceptually replicating the perimeter across three consecutive virtual copies.
Let the posts be extended periodically such that $p_{i+tn} = p_i + tL$ for all index i and $t \in \{-1, 0, 1\}$, meaning index i, $i - n$, and $i + n$ denote the same physical post. Under this framework, $t = -1$ shifts the coordinates backward by one full circle length (positions from $-L$ to 0), $t = 0$ maintains the original coordinates (positions from 0 to L), and $t = 1$ shifts them forward (positions from L to $2L$). By creating this extended array of size

$3n$, any transmitter position $c \in [0, L]$ can look strictly forward and backward within its radius r without running out of bounds. For a real number $x \in [-L, L]$ define

$$\text{lo}(x) = \min\{i : p_i \geq x - r\}, \qquad \text{hi}(x) = \max\{i : p_i \leq x + r\}.$$

These give the first and last posts covered by the transmitter.

(A) If $x \leq y$ then $\{i : p_i \geq y - r\} \subseteq \{i : p_i \geq x - r\} \implies \text{lo}(x) \leq \text{lo}(y)$. Similarly, $\{i : p_i \leq x + r\} \subseteq \{i : p_i \leq y + r\} \implies \text{hi}(x) \leq \text{hi}(y)$.Both are non-decreasing in x.

(B) Since $p_{i+tn} = p_i + tL$, we have $p_i \geq x + tL - r \iff p_{i-tn} \geq x - r$. Hence i belongs to the set defining $\text{lo}(x + tL)$ iff $i - tn$ belongs to the set defining $\text{lo}(x)$, so $\text{lo}(x + tL) = \text{lo}(x) + tn$. The same shift argument gives $\text{hi}(x + tL) = \text{hi}(x) + tn$.

A simple binary search using `bisect_left` and `bisect_right` evaluates these boundaries, finding the starting and ending indices $\text{lo}(x)$ and $\text{hi}(x)$ as a plain, continuous linear interval. The unrolled indices served by a site at x are exactly $[\text{lo}(x), \text{hi}(x)]$. Since $2r < L$ this window contains each physical post at most once, so it has at most n indices.

Since every transmitter's interval is now linear, we can define its absolute coverage arc (s, e) inside our standard window. For a candidate in transmitters with center c that serves at least one post, let $t \in \{-1, 0, 1\}$ be the unique integer such that $0 \leq \text{lo}(c+tL) < n$ and define its *arc* $(s, e) = (\text{lo}(c+tL), \text{hi}(c+tL))$. By finding this unique shifting factor t, we force the start index s to sit safely within the primary period ($0 \leq s < n$), while letting the end index e extend past n if it wraps around the perimeter boundary.

The arc covers the posts with indices $s, s + 1, \ldots, e$ (taken modulo n). Its length is $e - s + 1 \leq n$. Candidates serving no post are discarded; two candidates with the same arc are interchangeable, so we keep one representative per distinct arc. If some arc has length n, a single site serves all posts and the number of check-posts that have more than one transmitters is 0 and that site is an optimal location. Otherwise, every arc has length at most $n - 1$.

Let the number $\tilde{c} = c + tL$ be the *effective centre* of the arc.

Lemma 1 (Monotonicity of arcs)**.** *Let (s, e) and (s', e') be arcs with $0 \leq s, s' < n$, and effective centres $\tilde{c}, \tilde{c}'$ (so $s = \text{lo}(\tilde{c})$, $e = \text{hi}(\tilde{c})$, $s' = \text{lo}(\tilde{c}')$, $e' = \text{hi}(\tilde{c}')$).*

1. *If $s < s'$, then $\tilde{c} < \tilde{c}'$ and $e \leq e'$. Hence, after sorting the arcs by (s, e) lexicographically, both s and e are non-decreasing along the list.*
2. *$e' - n \leq e$ (for* every *pair of arcs).*

Proof. (1) Let $s < s'$, i.e. $\text{lo}(\tilde{c}) < \text{lo}(\tilde{c}')$. As lo is non-decreasing, $\tilde{c} < \tilde{c}'$, and then (A) gives $e = \text{hi}(\tilde{c}) \leq \text{hi}(\tilde{c}') = e'$. For sorted list, consecutive arcs $(s_1, e_1), (s_2, e_2)$ with $(s_1, e_1) < (s_2, e_2)$ lexicographically either have $s_1 < s_2$, in which case $e_1 \leq e_2$ by what was just shown, or $s_1 = s_2$ and $e_1 \leq e_2$ by the sort order. In both cases $s_1 \leq s_2$ and $e_1 \leq e_2$.

(2) (B), $\text{lo}(\tilde{c}' - L) = s' - n < 0 \leq s = \text{lo}(\tilde{c})$, using $s' < n$ and $s \geq 0$. As, lo is non-decreasing $\tilde{c}' - L < \tilde{c}$, and by (A) and (B), $e' - n = \text{hi}(\tilde{c}' - L) \leq \text{hi}(\tilde{c}) = e$. □ □

Corollary 1. If two distinct arcs of the list satisfy $[s, e] \subseteq [s', e']$ as index intervals, then $s = s'$ or $e = e'$.

Proof. Suppose $s' < s$. By Lemma 1(1) applied to (s', e') and (s, e), $e' \leq e$, together with $e \leq e'$ this gives $e = e'$. Otherwise $s' \geq s$, and $[s, e] \subseteq [s', e']$ forces $s' = s$. □ □

Since all transmitters have the same radius, if one arc is contained in another, their endpoints must share at least one boundary. Therefore, removing an arc simply because it is contained in another is not always safe: replacing it with the larger arc may increase the number of posts served by multiple transmitters.

2.2 Chains

A *chain* is a sequence of intervals/arcs $A_1, \ldots, A_k$ ($A_i = (s_i, e_i)$) with

(C1) $s_1 < s_2 < \cdots < s_k$ and $e_1 < e_2 < \cdots < e_k$ (strict progress),

(C2) $s_{i+1} \leq e_i + 1$ for $i < k$ (no gap),

(C3) $e_k \geq s_1 + n - 1$ (the last arc closes the circle).

A chain is defined as minimal if every single transmitter before the last one ($i < k$) fails to close the circle on its own (i.e., $e_i < s_1 + n - 1$), ensuring no redundant loops or over-travel are introduced. By (C2) and (C3), the unrolled intervals of a chain cover $[s_1, s_1 + n - 1]$, so a chain is a feasible solution. Define its *chain cost*

$$S(C) = \sum_{i=1}^{k-1} \big(e_i - s_{i+1} + 1\big) \; + \; \big(e_k - s_1 - n + 1\big).$$

Let $T_i = \{s_{i+1}, \ldots, e_i\}$ for $i < k$ and $T_k = \{s_1, \ldots, e_k - n\}$ be index sets, which are possibly empty when $s_{i+1} > e_i$ and $e_k - n < s_1$ respectively, and whose elements are strictly increasing; then $S(C) = \sum_i |T_i|$. Intuitively, T_i is the set of posts shared by the cyclically consecutive arcs A_i, A_{i+1}.

3 Algorithm 1

We now present an algorithm of complexity $\mathcal{O}(m^3 + m \log n + n \log n)$. To find a minimal chain that minimizes the total chain cost $S(C)$, we break the circular loop by treating each unique arc, one by one, as a fixed starting anchor for a straight-line sequence. Fixing an anchor changes the circular layout into a simple linear path optimization. This allows us to use dynamic programming to systematically build the chain forward under the strict structural rules established in Section **2.2**.

Let $\mathcal{A} = \{A_0, A_1, \ldots, A_{M-1}\}$ be the set of unique representative arcs, sorted such that $s_0 \le s_1 \le \cdots \le s_{M-1}$ and consequently $e_0 \le e_1 \le \cdots \le e_{M-1}$ according to Lemma 1.

3.1 State Definition

Fix an anchor arc index $a \in \{0, \ldots, M-1\}$ with $A_a = (s_a, e_a)$. For any subsequent arc $A_i = (s_i, e_i)$ with $i > a$ and $s_i > s_a$, we define $dp[i]$ as the minimum accumulated overlap cost of a valid partial chain originating at A_a and terminating at A_i, satisfying conditions (C1) and (C2) over the unrolled segment up to e_i.

3.2 Recurrence Relation

The base case begins at the designated anchor state with zero initial penalty:

$$dp[a] = 0 \tag{1}$$

For each subsequent interval A_i ($i > a$), a preceding interval A_j ($a \le j < i$) can validly extend the chain if it leaves no structural gaps, satisfying condition (C2). The algorithm strictly enforces this continuity requirement by validating the following condition during each state transition:

$$e_j \ge s_i - 1 \tag{2}$$

When this condition holds, the overlap cost contribution of transitioning from A_j to A_i is given by $e_j - s_i + 1$. The dynamic programming recurrence relation evaluates the optimal predecessor j that minimizes the running chain cost:

$$dp[i] = \min_{\substack{a \le j < i \\ e_j \ge s_i - 1}} \Big(dp[j] + e_j - s_i + 1 \Big) \tag{3}$$

By taking out the invariant terms depending purely on the choice of j inside the minimization loop, we optimize this step algebraically as:

$$dp[i] = \left(\min_{\substack{a \le j < i \\ e_j \ge s_i - 1}} [dp[j] + e_j] \right) - s_i + 1 \tag{4}$$

If an arc A_i has $s_i = s_a$, it cannot validly extend a chain originating from A_a under the strict progress condition (C1) and is skipped during the transition phase.

3.3 Cycle Closure and Global Minimization

A partial chain terminating at index i successfully closes the circular loop back to the anchor if it satisfies condition (C3). This occurs when the unrolled span of the chain reaches or extends past the anchor's wrap-around boundary:

$$e_i \ge s_a + n - 1 \tag{5}$$

When completing a valid loop, any final over-coverage between the tail arc A_i and the head anchor A_a adds a final cyclic penalty $(e_i - s_a - n + 1)$. The total chain cost for the configuration is:

$$\text{Cost}(a, i) = dp[i] + e_i - s_a - n + 1 \tag{6}$$

The globally optimal configuration minimizing the total multi-coverage interference across the entire perimeter is obtained by finding the absolute minimum across all possible anchor choices:

$$\text{Optimal Cost} = \min_{0 \le a < M} \min_{\substack{a \le i < M \\ e_i \ge s_a + n - 1}} \text{Cost}(a, i) \tag{7}$$

4 Source Code of Implementation

The full Python implementation of the circular dynamic programming algorithm, incorporating binary-search interval extraction and chain optimization, is detailed below. The Python implementation is also available on Github: *https : //github.com/shyamalborgohain/DAAproject/tree/main/Pb*10

```python
from bisect import bisect_left, bisect_right
from math import inf

def optimal_transmitters(posts, L, candidates, r=250):

    if posts==[]:
        return 0, []
    if candidates==[]:
        return -1,None
    if 2 * r >= L:
        return 0, [candidates[0]]

    P = sorted(q % L for q in posts)
    n = len(P)

    # Unrolling the Circle into Linear Arcs
    extP = [q - L for q in P] + P + [q + L for q in P]
    locs = {}
    # Finding the range of check posts covered by each candidate transmitter
    for ca in candidates:
        c = ca % L
        lo = bisect_left(extP, c - r)
        hi = bisect_right(extP, c + r) - 1
        if lo > hi:
            continue

        # making sure that 0<=lo<n
        if lo >= n: lo -= n; hi -= n
        elif lo < 0: lo += n; hi += n

        if hi - lo + 1 >= n:
            return 0, [ca]
        locs.setdefault((lo, hi), ca)

    arcs = sorted(locs) # interval/arc is the set of ordered checkpost covered by
    transmitters
    M = len(arcs)
    if M == 0:
        return -1,None

    start = [arc[0] for arc in arcs]
    end = [arc[1] for arc in arcs]

    best, best_chain = inf, None

    # O(M^3) DP Loops
    for a in range(M): # Fix the starting interval of the chain
        sa = start[a]
        dp = [inf] * M
        dp[a] = 0
        par = [-1] * M

        for i in range(a + 1, M):  # i as the last interval in the
                                   # current sub-chain under anchor a
            if start[i] == sa:
                continue
```

```
            best_val = inf
            best_j = -1

            # Loop through all previous arcs in the current chain
            for j in range(a, i):
                if dp[j] < inf: # Check if a valid, unbroken chain
                                # exists from anchor a to j

                    if end[j] >= start[i] - 1:# ensure arc j reaches far enough
                                              # to touch/overlap arc i (No gaps)
                        val = dp[j] + end[j]
                        if val < best_val:
                            best_val = val
                            best_j = j

            if best_val < inf: # Updating best_val and parent for chain where a
                               # is the anchor and i is the last interval
                dp[i] = best_val - start[i] + 1
                par[i] = best_j

        #Circle Wrap-Around Verification
        for i in range(a, M):
            # Verify if the chain closes
            # the circular loop back to anchor a
            if dp[i] < inf and end[i] >= sa + n - 1:
                cost = dp[i] + (end[i] - sa - n + 1)
                if cost < best:
                    best = cost
                    # Backtrack to reconstruct the optimal chain sequence
                    chain = [] # Temporary list storing the parent indices
                               # of the current valid loop
                    k = i
                    while k != -1:
                        chain.append(k)
                        k = par[k]
                    best_chain = chain[::-1] # Optimal sequence of intervals
                                             # for the current anchor "a"

    if best == inf:
        return -1,None

    return best, [locs[arcs[k]] for k in best_chain]

if __name__ == "__main__":
    posts = [100, 400, 700, 900]
    L = 1000
    candidates = [50, 350, 650, 950]
    print(*optimal_transmitters(posts, L, candidates, r=250))
```

Listing 1: Optimal Transmitter Placement

5 Proof of Correctness

The proof of correctness proceeds in two stages. Firstly, we validate the chain by showing that any solution sequence reconstructed via backtracking constitutes a valid, complete circular coverage chain. Secondly, we establish *completeness and optimality* by proving that the algorithm identifies a configuration that achieves the true global minimum cost.

Proof. Let $\mathcal{A} = \{A_0, A_1, \ldots, A_{M-1}\}$ be the set of unique candidate arcs sorted such that $s_0 \leq s_1 \leq \cdots \leq s_{M-1}$ and $e_0 \leq e_1 \leq \cdots \leq e_{M-1}$.

5.1 Chain Validity

Let $A_{k_1}, A_{k_2}, \ldots, A_{k_m}$ be a sequence of arcs reconstructed via backtracking from a terminating state $i = k_m$ for a fixed anchor $a = k_1$. We show it satisfies conditions (C1), (C2), and (C3):

- **Strict Progress (C1):** Arcs are sorted by s, and transitions enforce $s_j < s_i$. If two arcs are nested (e.g., $A_i \subset A_j$ where $e_i = e_j$), either may be chosen independently, but never both. Transitioning through both to some A_k adds a cost of $\Delta\text{Cost}_{\text{nested}} = (e_j - s_i + 1) + (e_j - s_k + 1)$, whereas a direct jump $A_j \to A_k$ costs $\Delta\text{Cost}_{\text{direct}} = e_j - s_k + 1$. The difference is $e_j - s_i + 1 \geq 1$ since a valid arc A_i must cover at least one post ($s_i \leq e_j$). Because the direct transition is strictly cheaper, the cost-minimization framework naturally avoids coexisting nested arcs, satisfying strict progress ($s_j < s_i$ and $e_j < e_i$).
- **No Gaps (C2):** The transition constraint $e_j \geq s_i - 1$ guarantees continuity by induction. The base case $m = 1$ is trivially the single interval $[s_{k_1}, e_{k_1}]$. If a chain of size $m - 1$ forms the contiguous span $[s_{k_1}, e_{k_{m-1}}]$, the algorithm explicitly checks and enforces the constraint $e_{k_{m-1}} \geq s_{k_m} - 1$, ensuring that the next interval is adjacent to or overlaps it. Since Lemma 1 guarantees $e_{k_{m-1}} \leq e_{k_m}$, their union simplifies to $[s_{k_1}, e_{k_m}]$, preserving continuity without gaps.
- **Circle Closure (C3):** This condition is explicitly checked by the algorithm during the finalization phase; the evaluation routine records a valid loop closure if and only if $e_{k_m} \geq s_{k_1} + n - 1$. This check guarantees that the total unrolled coverage interval $[s_{k_1}, e_{k_m}]$ has a length of at least n. Since the coordinates are periodic modulo n, mapping these indices back to the primary period yields complete coverage of all n check-posts.

5.2 Completeness and Optimality

For a fixed anchor A_a, let $dp[i]$ be the value defined by the recurrence relation for $i > a$. We prove by induction on the chain size that $dp[i]$ equals the minimum total overlap cost of a valid partial chain spanning from A_a to A_i.

Base Case: For a chain consisting only of the anchor arc A_a, the state is initialized to $dp[a] = 0$. This configuration is optimal because an isolated arc contains no interior pairwise transitions, meaning its accumulated transition cost is identically zero.

Inductive Step: Assume that for all j such that $a \leq j < i$, $dp[j]$ correctly stores the minimum interior overlap cost for a partial chain terminating at A_j.

The number of check-posts multi-covered by the consecutive placement of A_j and A_i corresponds exactly to the cardinality of the intersection of their unrolled intervals. Because the recurrence relation explicitly enforces the condition $e_j \geq s_i - 1$ for a transition to be valid, and strict progress implies $s_j < s_i$, this intersection matches the index set $\{s_i, s_i + 1, \ldots, e_j\}$. The exact cost added by this transition is:

$$\text{Overlap}(j, i) = e_j - s_i + 1$$

To compute the minimum total cost $dp[i]$, the recurrence minimizes the sum of the optimal sub-chain cost up to arc j and the subsequent overlap penalty introduced by adding arc i, which is given via the recurrence:

$$dp[i] = \min_{\substack{a \leq j < i \\ e_j \geq s_i - 1}} \Big(dp[j] + e_j - s_i + 1 \Big) = \min_{\substack{a \leq j < i \\ e_j \geq s_i - 1}} \Big(dp[j] + e_j \Big) - s_i + 1 \tag{8}$$

If no valid predecessor j satisfies the structural criteria ($a \leq j < i$ and $e_j \geq s_i - 1$), or if all valid predecessors are themselves unreachable (where $dp[j] = \infty$), then state i is structurally unreachable from the current anchor A_a. In this case, the minimization domain evaluates to empty, yielding $dp[i] = \infty$.

Otherwise, because the terms s_i and 1 depend purely on the target state i, they remain invariant constants with respect to the minimization variable j. Factoring them outside the minimization operator transforms Equation (8) into:

$$dp[i] = \left(\min_{\substack{a \leq j < i \\ e_j \geq s_i - 1 \\ dp[j] < \infty}} [dp[j] + e_j] \right) - s_i + 1 \tag{9}$$

Consequently, this formulation evaluates the absolute minimum total cost required to form a valid partial chain from the designated anchor A_a to the target state A_i. By mathematical induction, this minimizing property holds across the sequence, guaranteeing that $dp[i]$ establishes the optimal total overlap cost for any valid subchain spanning between these two fixed boundaries, or yields $dp[i] = \infty$ if no such path exists.

Global Minimization Every valid circular chain contains a unique element that appears first under the lexicographical ordering of $\mathcal{A}$. Let this arc be A_a. Because the outer loop iterates over all possible choices of $a \in \{0, \ldots, M-1\}$, the search space comprehensively includes the exact anchor that originates the globally optimal chain C^*.

When a chain closes at a valid terminal arc A_i, the wrap-around multi-coverage between the tail arc A_i and the anchor A_a occurs where the unrolled indices of the final arc intersect the forward-shifted period of the anchor. This cyclic overlap domain is defined by the intersection of $[s_i, e_i]$ and $[s_a + n, e_a + n]$. Because condition (C3) requires $e_i \geq s_a + n - 1$, and minimal chain progress ensures $s_i \leq s_a + n$ while Lemma 1 guarantees $e_i \leq e_a + n$, this intersection reduces exactly to the unrolled index interval $[s_a + n, e_i]$, yielding a wrap-around penalty of:

$$\text{Wrap-around Cost} = e_i - (s_a + n) + 1 = e_i - s_a - n + 1$$

The total cost of the closed circular chain is exactly $S(C) = dp[i] + e_i - s_a - n + 1$. Minimizing this total over all combinations of $0 \leq a < M$ and $a < i < M$ ensures that the system identifies the global minimum cost.

□

6 Complexity Bounds for Algorithm 1

6.1 Time Complexity

- **Preprocessing and Mapping:** Sorting the check-posts requires $\mathcal{O}(n \log n)$ time. Extracting the coverage boundaries for the m transmitter candidates via binary search takes $\mathcal{O}(m \log n)$ time.
- **Dynamic Programming Phase:** The algorithm uses three nested loops. The outer loop selects the starting anchor a, the middle loop iterates over the target arc i, and the inner loop examines eligible predecessor arcs j. Each state transition requires $\mathcal{O}(1)$ time. Since there are at most M choices for the anchor, and the two inner loops require $\mathcal{O}(M^2)$ time per anchor, the total time complexity of the dynamic programming phase is $\mathcal{O}(M^3)$, where $M \leq m$ is the number of unique arcs.

The global time complexity bound is:

$$\mathcal{O}(M^3 + m \log n + n \log n) \subseteq \mathcal{O}(m^3 + m \log n + n \log n)$$

6.2 Space Complexity Analysis

The layout utilizes linear memory configurations:

- The unrolled periodic check-post array requires $\mathcal{O}(n)$ storage.
- The state vectors (*dp*, *par*) and coordinate arrays (*start*, *end*) scale linearly with the number of unique arcs M.

The global space complexity is strictly bounded at $\mathcal{O}(M + n) \subseteq \mathcal{O}(m + n)$.

7 An Optimized Version of Algorithm 1

To find a minimal chain that minimizes the total chain cost $S(C)$, we break the circular loop by treating each unique arc, one by one, as a fixed starting anchor for a straight-line sequence. Fixing an anchor changes the circular layout into a simple linear path optimization. This allows us to use **Sliding Window and Monotonic Deque**(a double-ended queue whose elements are maintained in either nondecreasing or nonincreasing order) along with **Dynamic Programming** to systematically build the chain forward under the strict structural rules established in Section 2 in $\mathcal{O}(M^2)$ overall time. This will reduce the overall complexity of the algorithm to $\mathcal{O}(M^2 + m \log n + n \log n)$.

Let $\mathcal{A} = \{A_0, A_1, \ldots, A_{M-1}\}$ be the set of unique representative arcs, sorted such that $s_0 \leq s_1 \leq \cdots \leq s_{M-1}$ and consequently $e_0 \leq e_1 \leq \cdots \leq e_{M-1}$ according to Lemma 1.

7.1 State Definition

Fix an anchor arc index $a \in \{0, \ldots, M-1\}$ with $A_a = (s_a, e_a)$. For any subsequent arc $A_i = (s_i, e_i)$ with $i > a$ and $s_i > s_a$, we define $dp[i]$ as the minimum accumulated overlap cost of a valid partial chain originating at A_a and terminating at A_i, satisfying conditions (C1) and (C2) over the unrolled segment up to e_i.

7.2 Recurrence Relation and Search Boundaries

The base case begins at the designated anchor state with zero initial penalty:

$$dp[a] = 0 \tag{10}$$

For each subsequent interval A_i ($i > a$), a preceding interval A_j ($a \leq j < i$) can validly extend the chain if it leaves no structural gaps, satisfying condition (C2). The algorithm strictly enforces this continuity requirement by validating the following condition during each state transition:

$$e_j \geq s_i - 1 \tag{11}$$

When this condition holds, the overlap cost contribution of transitioning from A_j to A_i is given by $e_j - s_i + 1$. The dynamic programming recurrence relation evaluates the optimal predecessor j that minimizes the running chain cost:

$$dp[i] = \min_{\substack{a \leq j < i \\ e_j \geq s_i - 1}} \Big(dp[j] + e_j - s_i + 1 \Big) \tag{12}$$

By taking out the invariant terms depending purely on the choice of j inside the minimization loop, we optimize this step algebraically as:

$$dp[i] = \left(\min_{L_i \leq j \leq R_i} [dp[j] + e_j] \right) - s_i + 1 \tag{13}$$

where the valid transition window boundaries are defined as:

$$L_i = \max(a, \text{lo_idx}[i]) \quad \text{with} \quad \text{lo_idx}[i] = \min\{k \mid e_k \geq s_i - 1\} \tag{14}$$

$$R_i = \text{hi_idx}[i] \quad \text{with} \quad \text{hi_idx}[i] = \min(\text{sg}[i], \text{eg}[i]) - 1 \tag{15}$$

Here, $\text{sg}[i]$ and $\text{eg}[i]$ indicate the first indices in $\mathcal{A}$ sharing the exact same start and end positions as A_i, respectively. If an arc A_i has $s_i = s_a$, it cannot validly extend a chain originating from A_a under the strict progress condition (C1) and is skipped during the transition phase.

7.3 Optimization using Sliding Window and Monotonic Deque

To evaluate Equation 13 in $\mathcal{O}(1)$ amortized time per state i, the window minimization is accelerated using a sliding-window and a double-ended queue, dq, and an ingestion pointer `ptr` initialized to a. The state transition executes via three sequential phases:

1. **Monotonic Ingestion (Push Phase):** The pointer `ptr` advances forward up to the upper boundary R_i. For each valid state where $dp[\texttt{ptr}] < \infty$, its potential metric is computed as $val = dp[\texttt{ptr}] + e_{\texttt{ptr}}$, where e_{ptr} is the end point of the ptr^{th} interval. Elements are iteratively popped from the tail of deque dq while $\text{dq}[-1][0] \geq val$ to maintain a strictly increasing cost order. The pair $(val, \texttt{ptr})$ is then appended to the tail of deque dq, and `ptr` increments by 1.

2. **Boundary Purge (Pop-Left Phase):** The lower window boundary is enforced as $L_i = \max(a, \text{lo_idx}[i])$. Elements at the front head of deque dq are permanently removed via left-popping operations while the stored indices fall outside the valid range of predecessors for i^{th} interval, i.e, ($\text{dq}[j][1] < L_i$ leading to the failure of the condition $e_j \geq s_i - 1$), as their endpoints can no longer bridge the coverage gap to $s_i - 1$.

3. **State Evaluation:** If deque dq is non-empty, the absolute structural minimum resides at the front head since deque dq is strictly increasing. The state value and its parent routing backpointer are assigned in $\mathcal{O}(1)$ time via:

$$dp[i] = \text{dq}[0][0] - s_i + 1 \tag{16}$$

$$par[i] = \text{dq}[0][1] \tag{17}$$

   If dq is empty, no continuous path reaches interval i under anchor a, and $dp[i] = \infty$.

7.4 Cycle Closure and Global Minimization

A partial chain terminating at index i successfully closes the circular loop back to the anchor if it satisfies condition (C3). This occurs when the unrolled span of the chain reaches or extends past the anchor's wrap-around boundary:

$$e_i \geq s_a + n - 1 \tag{18}$$

When completing a valid loop, any final over-coverage between the tail arc A_i and the head anchor A_a adds a final cyclic penalty $e_i - s_a - n + 1$. The total chain cost for the configuration is:

$$\text{Cost}(a, i) = dp[i] + e_i - s_a - n + 1 \tag{19}$$

The globally optimal configuration minimizing the total multi-coverage interference across the entire perimeter is obtained by finding the absolute minimum across all possible anchor choices:

$$\text{Optimal Cost} = \min_{0 \le a < M} \min_{\substack{a \le i < M \\ e_i \ge s_a + n - 1}} \text{Cost}(a, i) \tag{20}$$

8 Source Code of Implementation

The full Python implementation of the circular dynamic programming algorithm, incorporating binary-search interval extraction and chain optimization, is detailed below. The Python implementation is also available on Github: *https : //github.com/shyamalborgohain/DAAproject/tree/main/optimized_Pb*10.

```python
from bisect import bisect_left, bisect_right
from collections import deque
from math import inf

def optimal_transmitters(posts, L, candidates, r=250):

    if posts==[]:
        return 0, []
    if candidates==[]:
        return -1,None
    if 2 * r >= L:
        return 0, [candidates[0]]

    P = sorted(q % L for q in posts)
    n = len(P)

    # Unrolling the Circle into Linear Arcs
    extP = [q - L for q in P] + P + [q + L for q in P]
    locs = {}
    # Finding the range of check posts covered by each candidate transmitter
    for ca in candidates:
        c = ca % L
        lo = bisect_left(extP, c - r)
        hi = bisect_right(extP, c + r) - 1
        if lo > hi:
            continue

        # making sure that 0<=lo<n
        if lo >= n: lo -= n; hi -= n
        elif lo < 0: lo += n; hi += n

        if hi - lo + 1 >= n:
            return 0, [ca]
        locs.setdefault((lo, hi), ca)

    arcs = sorted(locs) # interval/arc is the set of ordered checkpost covered by
                        # transmitters
    M = len(arcs)
    if M == 0:
        return -1, None

    start = [arc[0] for arc in arcs]
    end = [arc[1] for arc in arcs]
    # Pre-calculate the first index where each unique start/end value appears
    sg, eg = [0]*M, [0]*M
    for i in range(1, M):
        # sg["i"] and eg["i"] indicate the first indices in arcs sharing
        # the exact same start and end positions as arc "i", respectively
        sg[i] = sg[i-1] if start[i] == start[i-1] else i
        eg[i] = eg[i-1] if end[i] == end[i-1] else i

```

```
    # lo_idx["i"]: First interval ending close enough to start["i"] to avoid
    # coverage gaps
    lo_idx = [bisect_left(end, start[i]-1) for i in range(M)]

    # hi_idx["i"]: Latest interval that can safely link to "i" without
    # any endpoints in common
    hi_idx = [min(sg[i], eg[i]) - 1 for i in range(M)]

    best, best_chain = inf, None
    for a in range(M): # Fix the starting interval of the chain at index "a"
        sa = start[a]
        dp = [inf]*M # dp["i"]: Minimum penalty to cover from anchor "a" up
                     # to interval "i" first set to inf
        dp[a] = 0
        par = [-1]*M # Parent pointers for path reconstruction

        dq = deque() # Monotonic queue tracking pairs of
                     # (transition_value, interval_index)
        ptr = a
        for i in range(a+1, M): # Evaluate interval "i" as the last interval
                                # in the current sub-chain under anchor "a"
            if start[i] == sa:
                continue
            while ptr <= hi_idx[i]: # Queue candidates whose endpoints
                                    # do not match interval "i"
                if dp[ptr] < inf: # checking reachability before queuing
                    val = dp[ptr] + end[ptr]
                    while dq and dq[-1][0] >= val:
                        # Maintain a strictly ascending cost order by removing
                        # sub-optimal options from the tail of dq
                        dq.pop()
                    dq.append((val, ptr))
                ptr += 1

            lo = max(a, lo_idx[i]) # Remove out-of-bounds intervals from the front
                                   # that create an uncovered gap
            while dq and dq[0][1] < lo:
                dq.popleft()
            if dq: # update the dp state from the minimum valid predecessor
                dp[i] = dq[0][0] - start[i] + 1
                par[i] = dq[0][1]

        #Circle Wrap-Around Verification
        for i in range(a, M):
        # Verify if the chain closes the circular loop back to anchor a
            if dp[i] < inf and end[i] >= sa + n - 1:
                cost = dp[i] + (end[i] - sa - n + 1)
                if cost < best:
                    best = cost
                    # Backtrack to reconstruct the optimal chain sequence
                    chain = [] # Temporary list storing the parent indices of the
                               # current valid loop
                    k = i
                    while k != -1:
                        chain.append(k)
                        k = par[k]
                    best_chain = chain[::-1] # Optimal sequence of intervals
                                             # for the current anchor "a"

    if best == inf:
        return -1,None

    return best, [locs[arcs[k]] for k in best_chain]

if __name__ == "__main__":
    posts = [100, 400, 700, 900]
    L = 1000
    candidates = [50, 350, 650, 950]
    print(*optimal_transmitters(posts, L, candidates, r=250))
```

Listing 2: Optimal Transmitter Placement

9 Proof of Correctness

Let $\mathcal{A} = \{A_0, A_1, \ldots, A_{M-1}\}$ be the set of unique representative transmitter arcs sorted lexicographically such that $s_0 \le s_1 \le \cdots \le s_{M-1}$ and $e_0 \le e_1 \le \cdots \le e_{M-1}$ (by Lemma 1). Let $C^* = (A_{k_1}, A_{k_2}, \ldots, A_{k_m})$ represent a globally optimal circular chain configuration.

To prove that the optimized algorithm is correct, we must establish two properties:

1. **Window Correctness:** The sliding window boundaries $[\max(a, \text{lo_idx}[i]), \text{hi_idx}[i]]$ accurately encapsulate all candidate predecessors that satisfy conditions (C1) and (C2) under anchor a.
2. **Monotonic Deque Optimality:** The double-ended queue elements successfully preserve the valid subset of reachable candidates and maintain the absolute optimal minimizer at the front head of the queue due to monotonicity.

9.1 Correctness of Window Boundaries

For a fixed anchor index a and a targeted interval A_i (where $i > a$ and $s_i > s_a$), an interval index j qualifies as a *valid linear predecessor* if and only if it satisfies the following structural constraints:

$$\text{(C1) Strict Progress: } s_j < s_i \quad \text{and} \quad a \le j < i \tag{21}$$

$$\text{(C2) No Gaps: } e_j \ge s_i - 1 \tag{22}$$

Lemma 2 (Monotonicity of Predecessor Search Space)**.** *For a target interval i under anchor a, the set of valid, non-redundant linear predecessors forms a single, contiguous integer interval $[L_i, R_i]$, where $L_i = \max(a, lo_idx[i])$ and $R_i = hi_idx[i]$.*

Proof. **Upper Bound (R_i)** Condition (C1) requires strict start-point progress ($s_j < s_i$). Additionally, to avoid redundant transition penalties due to arc containment, a valid predecessor must not share the target's endpoint ($e_j < e_i$). The algorithm sets $\text{hi_idx}[i] = \min(\text{sg}[i], \text{eg}[i]) - 1$, where $\text{sg}[i]$ and $\text{eg}[i]$ signify the first indices in $\mathcal{A}$ sharing the same start and end points as A_i, respectively.

Due to monotonic sorting, any index k where $\text{sg}[i] \le k \le i$ implies $s_k = s_i$, which explicitly violates the strict progress condition (C1). Similarly, any index k where $\text{eg}[i] \le k \le i$ implies $e_k = e_i$, meaning a transition from A_k to A_i contributes zero additional coverage while introducing an unnecessary multi-coverage penalty. Conversely, for any index $j \le \text{hi_idx}[i]$, global monotonicity guarantees that $s_j < s_i$ and $e_j < e_i$, successfully satisfying all validity and non-redundancy constraints. Thus, $R_i = \text{hi_idx}[i]$ establishes the exact upper limit.

Lower Bound (L_i) Condition (C2) requires that a predecessor leaves no structural gap ($e_j \ge s_i - 1$). The boundary $\text{lo_idx}[i]$ is isolated via a binary search on the sorted `end` array. Because the endpoints are monotonically non-decreasing, any index $k < \text{lo_idx}[i]$ is guaranteed to have $e_k < s_i - 1$, which breaks chain continuity and violates condition (C2).

Conversely, for any index $j \ge \text{lo_idx}[i]$, monotonicity ensures that $e_j \ge e_{\text{lo_idx}[i]} \ge s_i - 1$. Additionally, enforcing a progressive linear path execution from the fixed starting anchor A_a mandates that $j \ge a$ to prevent cyclic back-tracking. Intersecting these independent structural constraints yields the exact lower bound $L_i = \max(a, \text{lo_idx}[i])$, confirming that the valid search space is precisely bounded within the contiguous integer range $[L_i, R_i]$. □

9.2 Correctness and Completeness of Monotonic Deque Transitions

Let the cost function value associated with a predecessor state j be denoted as $val(j) = dp[j] + e_j$. For the algorithm to be correct, the dynamic programming array assignment must satisfy the identity:

$$dp[i] = \left(\min_{L_i \le j \le R_i} val(j) \right) - s_i + 1 \tag{23}$$

We establish this identity via mathematical induction on the inner target state index i.

Base Case When $i = a$, $dp[a] = 0$. An isolated anchor segment has no inner pairwise intersections, meaning its accrued transition overlap penalty is identically zero, matching the true structural cost.

Inductive Step Assume that for all states k where $a \leq k < i$, $dp[k]$ correctly reflects the minimum accumulated interior overlap cost of a valid partial chain originating at anchor a and terminating at A_k. To prove that the monotonic deque contains $\min_{L_i \leq j \leq R_i} val(j)$ at its head when state i is evaluated, we track the continuous maintenance of two structural invariants.

Ingestion Invariant (Push Phase) The tracking pointer `ptr` is initialized to a and advances sequentially up to hi_idx$[i]$. Because start points s and endpoints e are non-decreasing (Lemma 1), the upper window limit $R_i = \text{hi_idx}[i]$ shifts monotonically rightward as i increments ($R_i \leq R_{i+1}$). This allows `ptr` to maintain its position across successive target states instead of resetting, ensuring each candidate index $j \leq R_i$ is processed exactly once. Reachable elements $(val(j), j)$ are appended to the queue tail after popping any existing tail elements $(val(k), k)$ where $val(k) \geq val(j)$ and $k < j$. This removal is structurally valid because the lower boundary L_i also shifts monotonically rightward ($L_i \leq L_{i+1}$), meaning the older element k drops out of the valid window earlier than or at the same time as the newer element j. Since element j yields a lower (or equal) transition cost and remains valid longer, element k is completely redundant. Discarding these sub-optimal states maintains a strictly ascending cost sequence inside the queue ($\text{dq}[0][0] < \text{dq}[1][0] < \dots$), preserving the absolute minimum at the front head.

Boundary Purge Invariant (Pop-Left Phase) Before computing $dp[i]$, elements falling outside the lower window constraint must be removed from the front of the queue. Because the `end` array is non-decreasing, the lower boundary $L_i = \max(a, \text{lo_idx}[i])$ moves strictly rightward as i increases ($L_i \leq L_{i+1}$). If an element k at the front satisfies $k < L_i$, then $e_k < s_i - 1$, creating a gap that breaks connectivity for state i. Because L_i never moves backward, index k will cause a gap for all subsequent target states as well ($e_k < s_{i+t} - 1$ for all $t > 0$ as $k < L_i \leq L_{i+t}$). In other words, if an interval's endpoint is too short to reach the current state, it cannot reach any state appearing later in the sequence. Therefore, permanently dropping k is safe and never skips a valid choice.

Optimal Selection Following the push and pop-left operations, the queue contains exactly the valid subset of reachable predecessor indices $j \in [L_i, R_i]$, stripped of sub-optimal elements. Because the queue maintains elements in a strictly increasing cost sequence ($\text{dq}[k][0] < \text{dq}[m][0]$ for all internal queue positions $k < m$), the head element holds the absolute minimum value: $\text{dq}[0][0] = \min_{L_i \leq \text{dq}[j][1] \leq R_i} \text{dq}[j][0]$. If the queue is not empty, the assignment $dp[i] = \text{dq}[0][0] - s_i + 1$ precisely evaluates the optimized recurrence relation. Conversely, an empty queue implies that no valid predecessor can connect to A_i without a structural gap, leaving $dp[i] = \infty$. This completes the inductive step.

9.3 Global Optimality

Every valid minimal circular chain has a lexicographically first interval when sorted by coordinate bounds. Let this starting interval be A_a. Since the outer loop systematically evaluates every anchor $a \in \{0, 1, \dots, M-1\}$, the algorithm is guaranteed to check the exact starting anchor that originates the globally optimal chain C^*.

When a linear path successfully closes the loop at a boundary-spanning arc A_i satisfying $e_i \geq s_a + n - 1$, the final wrap-around overlap penalty matches:

$$\text{Wrap-around Cost} = e_i - s_a - n + 1 \tag{24}$$

The algorithm records the total configuration cost as $dp[i] + e_i - s_a - n + 1$. Because mathematical induction guarantees that $dp[i]$ tracks the exact minimal linear path cost up to terminal state i, taking the absolute minimum over all valid anchors a and cycle-closing terminals i ensures the system identifies the true global minimum, returning infinity (∞) if no such valid path exists.

10 Complexity Analysis

10.1 Time Complexity

- **Preprocessing and Mapping:** Sorting the checkposts requires $\mathcal{O}(n \log n)$ time, where $n = |posts|$. Extracting the coverage boundaries for the m transmitter candidates via binary search takes $\mathcal{O}(m \log n)$ time, where $m = |candidates|$. Sorting the unique arcs takes $\mathcal{O}(M \log M)$ time.
- **Dynamic Programming Phase:** The algorithm solves the linear subproblem by fixing a starting anchor interval a across M choices. For each anchor, the target state index i steps sequentially through the intervals. Crucially, the pointer ptr inside the inner queue extraction loop increments strictly forward without resetting across iterations of i. Because each interval is pushed and popped from the monotonic deque at most once, the inner operations take an amortized $\mathcal{O}(M)$ time per anchor. Thus, this phase takes $\mathcal{O}(M^2)$ time, where $M \leq m$ is the number of unique arcs.

The global time complexity bound is:

$$\mathcal{O}(M^2 + m \log n + n \log n) \subseteq \mathcal{O}(m^2 + m \log n + n \log n)$$

10.2 Space Complexity

The layout utilizes linear memory configurations:

- The unrolled periodic checkpost array requires $\mathcal{O}(n)$ storage.
- The state vectors (dp, par), lookups (lo_idx, hi_idx, sg, eg), coordinate arrays ($start$, end), and the monotonic deque scale linearly with the number of unique arcs M.

The global space complexity is strictly bounded at $\mathcal{O}(M + n) \subseteq \mathcal{O}(m + n)$.

11 Conclusion

In this report, we have addressed the problem of optimal transmitter placement on a circular road, aiming to minimize multi-coverage interference while ensuring that every check post is covered.

We presented two dynamic programming algorithms. The first has time complexity $\mathcal{O}(m^3 + m \log n + n \log n)$, while the optimized version uses a sliding window and monotonic deque to reduce it to $\mathcal{O}(m^2 + m \log n + n \log n)$. Both algorithms output the optimal selected transmitter locations along with the minimum number of overlapping check posts.

The correctness of the algorithms is established using dynamic programming invariants and induction over the states. We show that each state considers all feasible predecessor transitions, ensuring that the computed value is optimal. For the optimized algorithm, the monotonic deque preserves the minimum-cost eligible predecessor, allowing the same optimal transitions to be evaluated more efficiently. Finally, the cycle-closure condition ensures complete coverage around the circular road.